From requests per day to GPU hours

A buyer’s guide to sizing AI inference

By Mumpe Cydrone, October 2026. An independent work sample. All measurements were taken on a free GPU, and none are vendor figures.

Summary

Goal. Buyers of AI inference capacity often get a hardware recommendation without seeing how it was reached. This guide gives a five step method for turning a workload, counted in requests per day, into a capacity estimate. It also shows where each step can go wrong.

Approach. I wrote the method first, then tested its weakest assumptions with real measurements. On 26 October 2026 I ran two models on one NVIDIA T4 GPU: Qwen2.5 3B at FP16 and Qwen2.5 7B compressed to 4 bit. Each ran at batch sizes of 1, 4 and 8, three times per setting, with a 512 token prompt and a 256 token answer. I then compared every speed with the memory bandwidth ceiling that many sizing guides use.

Results.

- The ceiling is not the expectation. The 3B model reached 25% to 40% of its ceiling, and the 7B model 10% to 27%.
- Batching raises volume and costs responsiveness. At batch 8 the 3B model produced about 5 times the total output, while each user’s answer streamed 38% slower and the first token took 7.7 times longer to appear.
- Fitting in memory does not mean running fast. The 4 bit 7B model used less memory than the 3B model (5.4 GB against 6.2 GB), yet it generated 22% fewer tokens per second at batch 1.

What this means. Three habits protect a buyer. First, never accept a speed claim without the software, batch size and first token time behind it. Second, size for peak load, and check latency at the batch size you will actually run, not only total throughput. Third, ask for measured speed on your own model, because memory fit and speed are separate questions. The method in section 3 turns these habits into steps. One caution: these results come from one small GPU and unoptimized software. They show how to ask the right questions, not what any larger system will deliver.

Terms used in this guide

Term	Meaning
Token	A small piece of text, roughly three quarters of an English word.
Throughput	Tokens generated per second. Total throughput counts every user at once. Per stream throughput is what one user sees.
Batch	The number of requests the GPU processes together.
First token time	How long a user waits before the answer begins. Often shortened to TTFT.
Quantization	Storing a model's numbers with fewer bits, such as 4 bit, to save memory.
Ceiling	The speed limit set by memory bandwidth: bandwidth divided by model size. Also called the roofline estimate.

1. The sizing question

Four inputs decide most of a sizing answer: how many tokens you process each day, how many users you serve at the same time, how long those users will wait, and which model you run. Everything else follows from them.

Start by turning business volume into tokens per second. Take an assistant that handles 10,000 requests a day, where each request has 512 input tokens and 256 output tokens.

Quantity	Calculation	Result
Tokens per day, all	10,000 × 768	7,680,000
Output tokens per day	10,000 × 256	2,560,000
Average output speed needed	2,560,000 ÷ 86,400 seconds	29.6 tokens per second

Quantity	Calculation	Result
Peak output speed, assuming 3 times average	29.6 × 3	88.9 tokens per second

The peak factor of 3 is an assumption for this example. Measure your own traffic before you choose one. Average load hides your busiest hour, and your users feel the busiest hour.

Input processing adds a smaller load. Here it averages about 59 tokens per second (5,120,000 input tokens divided by 86,400 seconds).

2. What I measured

Setup

- Hardware: one NVIDIA Tesla T4 with about 15 GB of memory, on a free Kaggle notebook, on 26 October 2026.
- Models: Qwen2.5 3B Instruct at FP16 (weights use 6.17 GB) and Qwen2.5 7B Instruct compressed to 4 bit (5.44 GB).
- Workload: a 512 token prompt and a 256 token answer, at batch sizes of 1, 4 and 8.
- Method: Hugging Face Transformers, greedy decoding, a fixed answer length, one warm up run, then three timed runs per setting.
- Ceiling: memory bandwidth of 320 GB/s divided by weight size, multiplied by batch size.

Setting	Ceiling (tokens per second)	Measured total (tokens per second)	Share of ceiling	First token time	Per stream (tokens per second)
3B FP16, batch 1	51.8	20.5	39.6%	0.20 s	20.5
3B FP16, batch 4	207.4	71.5	34.5%	0.73 s	17.9
3B FP16, batch 8	414.8	101.9	24.6%	1.53 s	12.7
7B 4 bit, batch 1	58.8	16.0	27.2%	0.54 s	16.0
7B 4 bit, batch 4	235.2	29.8	12.7%	1.70 s	7.4

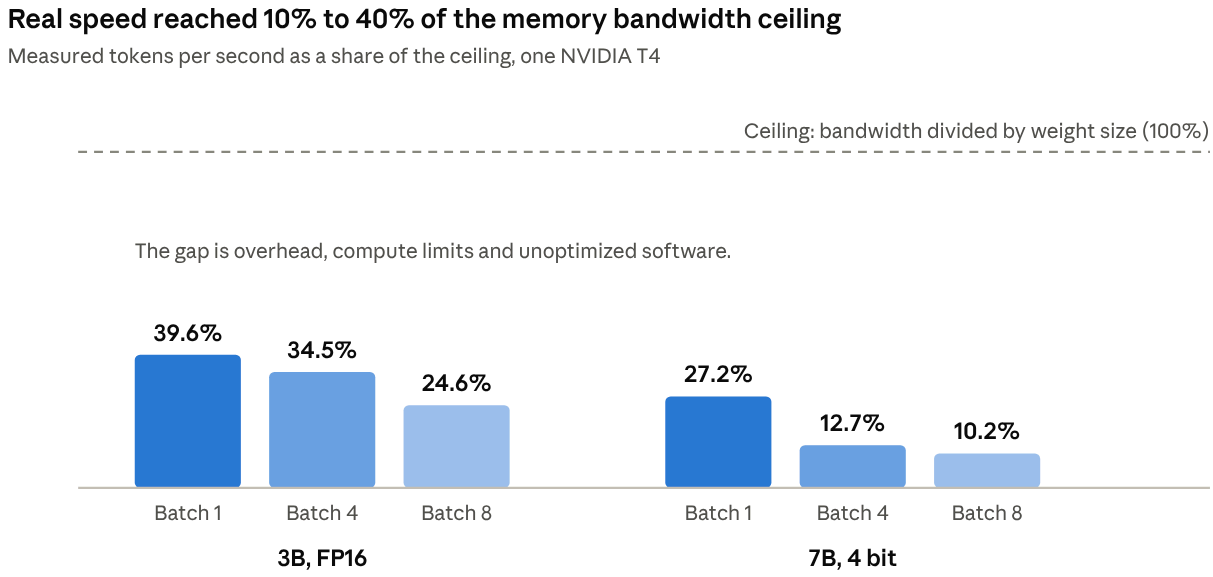
Setting	Ceiling (tokens per second)	Measured total (tokens per second)	Share of ceiling	First token time	Per stream (tokens per second)
7B 4 bit, batch 8	470.3	48.0	10.2%	3.34 s	6.0

Run to run spread was small. It stayed within about 3% in every setting, and under 2% in all but one.

Finding 1: The ceiling is an upper bound, not a forecast

When a model writes text, every new token requires reading all of the model's weights from memory. Memory bandwidth divided by weight size therefore gives a speed limit. A T4's bandwidth is commonly published as 320 GB/s, and the 3B model's weights take 6.17 GB. The ceiling for one user is 320 ÷ 6.17, or 51.8 tokens per second. I measured 20.5.

Software overhead, compute limits and unoptimized code all sit between the ceiling and real speed. In my tests the shares ranged from 10% to 40%, and they fell as batch size grew. Production serving engines usually close part of that gap, but a buyer should never assume the gap is zero. Ask any provider what share of the ceiling their quoted speed represents.



inference_benchmark_summary.csv, Kaggle NVIDIA T4, 26 Oct 2026, medians of 3 runs

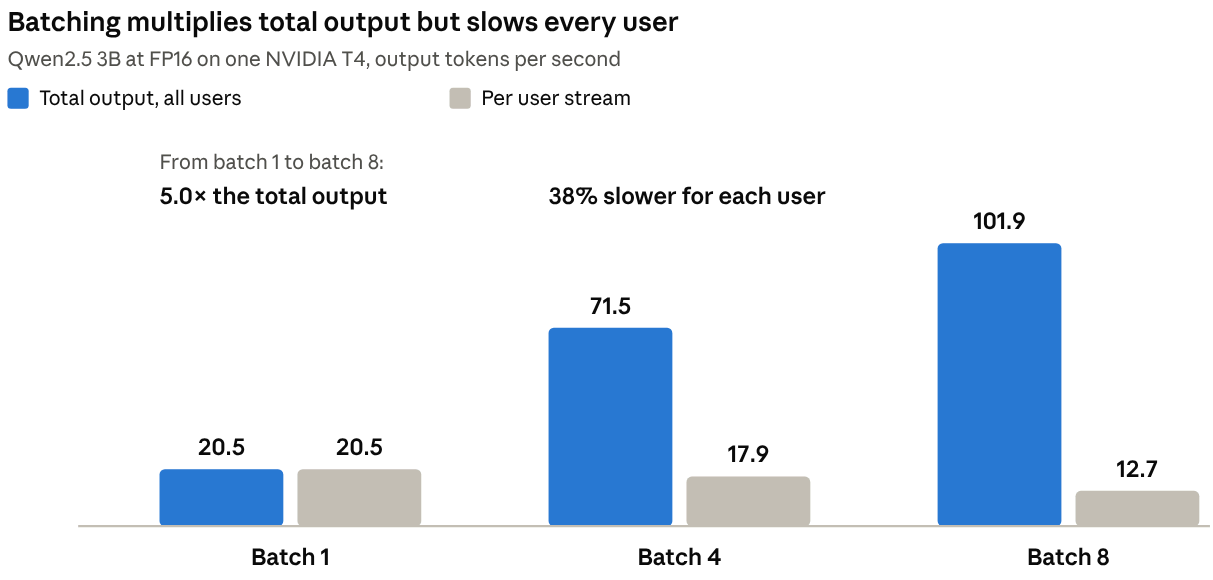
Figure 1. Share of the ceiling reached, by setting.

Finding 2: Batching raises volume and costs responsiveness

Batching lets the GPU read the weights once for several users at the same time, so total output rises. The price is paid by each user. This table shows the 3B model, with the time a user waits for a full 256 token answer.

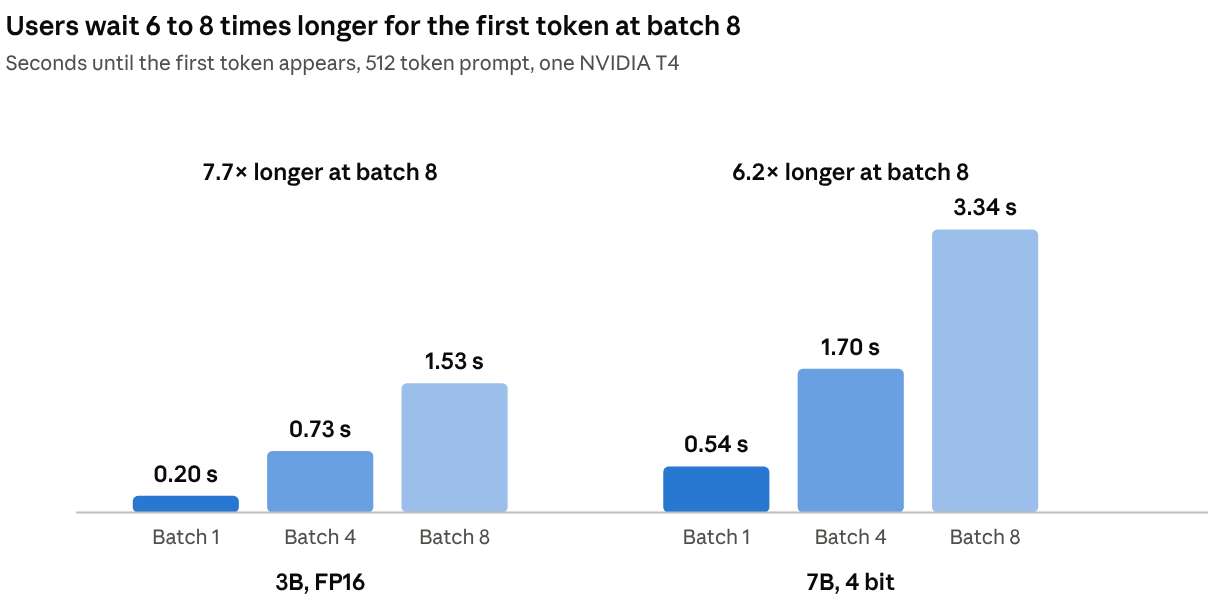
Batch	Total tokens per second	Per stream tokens per second	First token time	Time for a 256 token answer
1	20.5	20.5	0.20 s	12.7 s
4	71.5	17.9	0.73 s	15.1 s
8	101.9	12.7	1.53 s	21.6 s

From batch 1 to batch 8, total output rose about 5 times, each stream slowed by 38%, and the first token took 7.7 times longer. The 7B model showed the same pattern with larger penalties. Its total output rose 3.0 times, each stream slowed by 62%, and the first token took 6.2 times longer.



inference_benchmark_summary.csv, Kaggle NVIDIA T4, 26 Oct 2026, medians of 3 runs

Figure 2. Total and per stream speed by batch size, 3B model.



inference_benchmark_summary.csv, Kaggle NVIDIA T4, 26 Oct 2026, medians of 3 runs

Figure 3. First token time by batch size, both models.

Finding 3: Fitting in memory is not the same as being fast

Model	Weights in memory	First token time, batch 1	Tokens per second, batch 1
3B at FP16	6.17 GB	0.20 s	20.5
7B at 4 bit	5.44 GB	0.54 s	16.0

Compression let the larger model fit in less memory than the smaller one, yet it was 22% slower and took 2.75 times longer to reach the first token. Unpacking 4 bit numbers at every step is a likely cause, although I did not test it directly.

Compression is still valuable. The 7B model needs about 15 GB at full precision, which leaves no working room on a 16 GB card. The lesson is that fit and speed are two questions, and a buyer should answer both.

The example workload against the measurements

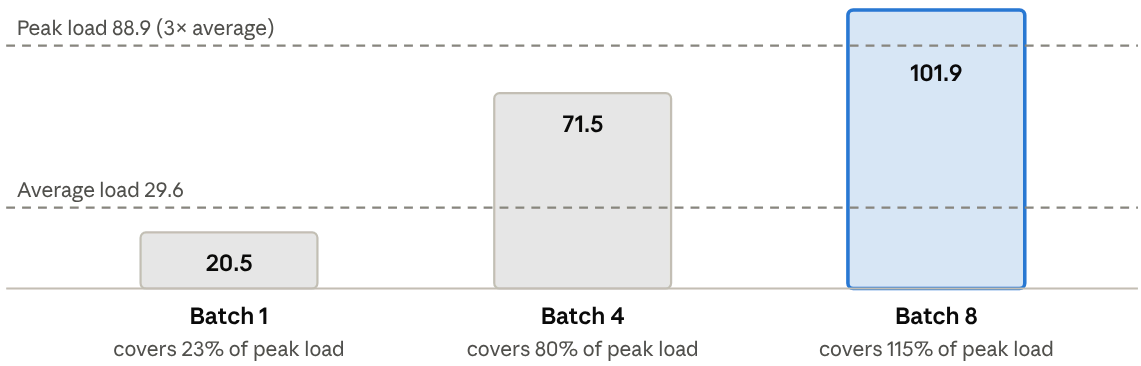
Take the workload from section 1 and compare it with what the 3B model delivered on one T4.

Batch	Capacity (tokens per second)	Average load as a share of capacity	Peak load as a share of capacity	Time for a 256 token answer
1	20.5	144%	433%	12.7 s
4	71.5	41%	124%	15.1 s
8	101.9	29%	87%	21.6 s

At batch 1 the GPU cannot keep up even with the average load. At batch 4 it handles the average but not the peak. At batch 8 it covers the peak with only 13% headroom, and users wait about 22 seconds for an answer. A buyer would likely need a second GPU or a faster setup, and would also have to decide whether a 3B model is accurate enough for the task. Input processing is a small cost: the average input load of 59 tokens per second compares with a measured speed of roughly 2,700 tokens per second, so it takes about 2% of the GPU's time. That figure is derived from first token times, so treat it as approximate.

Only batch 8 covers the peak load of the example workload

Output speed capacity of the 3B model on one NVIDIA T4, tokens per second



inference_benchmark_summary.csv and the section 1 example, Kaggle NVIDIA T4, 26 Oct 2026

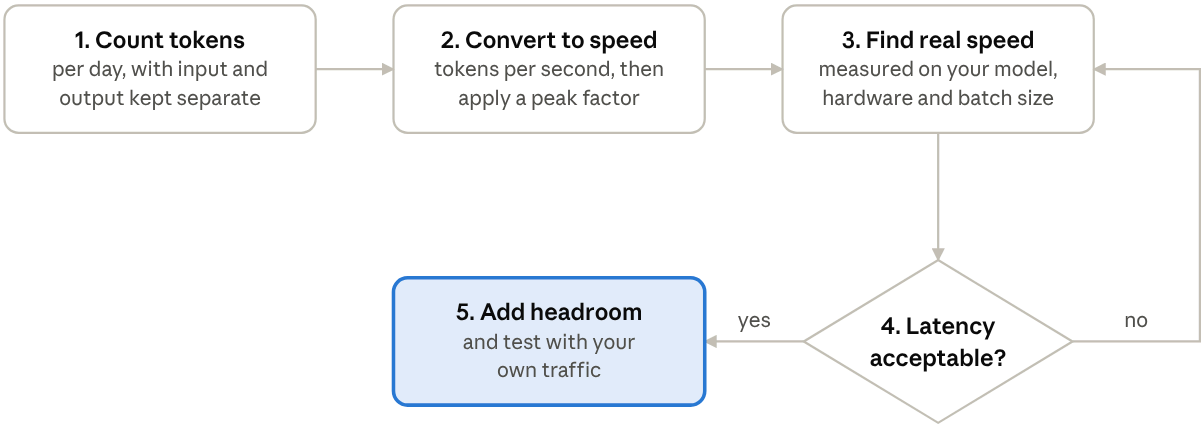
Figure 4. Capacity against the example workload, by batch size.

3. From measurement to a plan

Here is the method in five steps.

1. Count tokens per day, with input and output tokens kept separate.
2. Convert to tokens per second: divide by 86,400, then multiply by a peak factor.
3. Find measured throughput for your exact model, precision, hardware and serving software, at the batch size your latency target allows. If you only have a datasheet ceiling, multiply it by an efficiency factor and label that factor as an assumption. In my test the factor ranged from 0.10 to 0.40.
4. Check latency: first token time plus output tokens divided by per stream speed. Compare the result with what your users will accept.
5. Add headroom and test with your own traffic before you commit.

Sizing ends only when measured latency fits what users will accept



If latency is too high, change the batch size, model or hardware and measure again from step 3.

five step sizing method, 1 decision and 1 loop back

Figure 5. The five step sizing method.

Rent, buy or build

Option	Strength	Cost to watch
Rent GPU capacity	Fast to start, easy to change, no hardware to maintain	Paying for idle hours. Monthly cost often counts every hour, whether or not you send requests.
Buy hardware	Lower cost per token at steady, high use	Power, cooling, staff, and hardware that ages.

Option	Strength	Cost to watch
Build your own or a modular site	Control and scale	The longest lead time and the largest commitment.

Questions to ask any provider

1. Which serving software and version produced the quoted speed?
2. What batch size, prompt length and answer length were used?
3. What is the first token time at that batch size?
4. What share of the hardware's theoretical ceiling does the speed represent?
5. How is monthly cost counted: every hour, or only hours in use?
6. Are input tokens included in the price and in the throughput figure?

4. What this test does not tell you

- One GPU type. A T4 is a small, older card, and these results do not transfer to larger GPUs.
- Unoptimized software. Hugging Face Transformers is not a production serving engine. Optimized engines usually do better, so the shares of the ceiling here would likely be higher for them.
- A shared free tier. Other users may affect speed, although my three runs agreed closely.
- Three runs per setting, one fixed prompt repeated across the batch, and greedy decoding with a forced answer length. Real traffic varies.
- An assumed bandwidth of 320 GB/s, the commonly published figure for the T4. Check it against NVIDIA's datasheet before relying on the shares.
- First token time includes generation overhead, so the input processing speed in section 2 is approximate.
- I did not measure answer quality, power use or price. A small model that is fast may not be accurate enough for your task.

Appendix: Method and full results

Formulas

- Average tokens per second = tokens per day ÷ 86,400

- Ceiling for one stream = memory bandwidth ÷ weight size
- Ceiling for a batch = ceiling for one stream × batch size
- Share of ceiling = measured speed ÷ ceiling
- Time for an answer ≈ first token time + (output tokens ÷ per stream speed)

Full results

Setting	Total tokens per second, median (range)	Peak memory in use
3B FP16, batch 1	20.5 (19.9 to 20.6)	6.3 GB
3B FP16, batch 4	71.5 (71.0 to 71.5)	6.5 GB
3B FP16, batch 8	101.9 (101.7 to 102.1)	6.9 GB
7B 4 bit, batch 1	16.0 (15.9 to 16.0)	6.1 GB
7B 4 bit, batch 4	29.8 (29.6 to 30.1)	6.4 GB
7B 4 bit, batch 8	48.0 (48.0 to 48.0)	7.0 GB

Reproducibility. The benchmark notebook, the raw results (inference_benchmark_runs.csv and inference_benchmark_summary.csv), the environment record and my verification log are available on request. The test ran on 26 October 2026 on a Tesla T4 with driver 580.178.04.